Prototyping Project

Battery Savers

PHY5IL: Physical Prototyping

MTD.ba VZ WS25

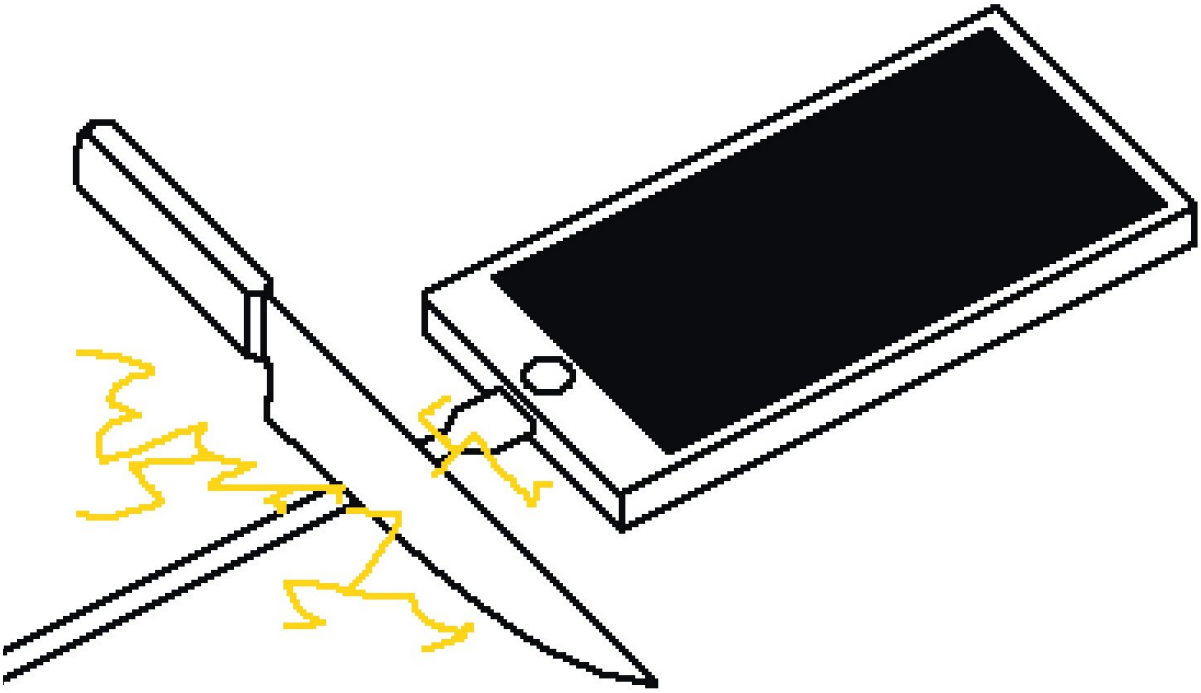
Team Members:

Maximilian Kriegner – s2310238086
Lara Eck - 2310238020

Project Premise

It has become common knowledge that you should not keep mobile electronic devices plugged in indefinitely, specifically mobile phones or tablets. Continuous overcharging of the device can cause damage to the battery and thus reduce its maximum charge capacity over time. While some more modern devices may feature a setting which only permits the device to charge partially (e.g. stops charging at 80%), some older devices may not have this ability.

The goal of this project is to remedy that shortcoming on older devices, by creating a device that can externally disconnect the power supply from a mobile device after a sufficient amount of charge has been acquired.



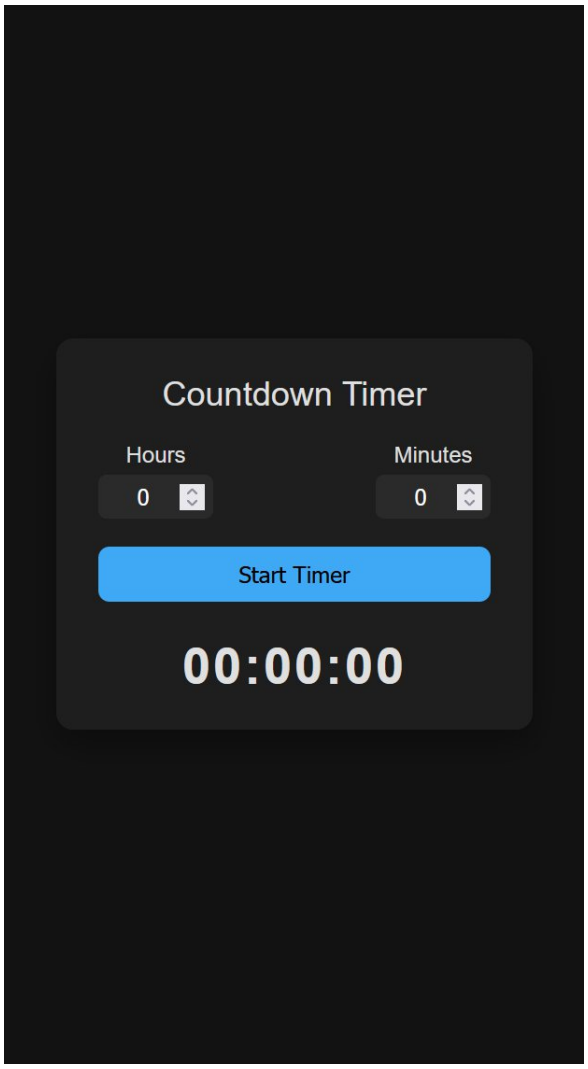
Design / code of prototype and Symbiosis Interpretation

The Symbiosis of the prototype stems from the fact that the physical device will be controlled via a web application accessed vie the mobile phone.

```
// WiFi credentials
const char* ssid = "EjectorHotspot";
const char* password = "EjectorPassword";
```

By default, the user has to enable a WIFI Hotspot with the name “EjectorHotspot” and password “EjectorPassword” using their phone. The device will then automatically connect to it and enable access to its web app.

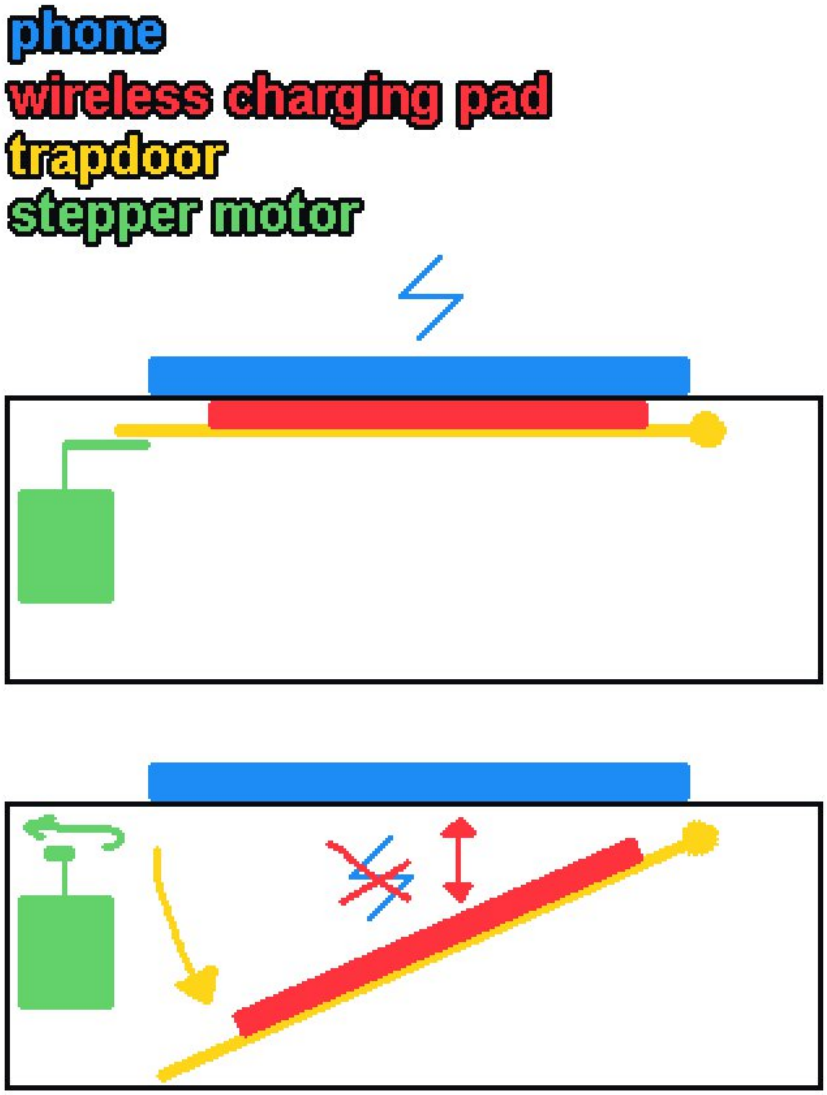
The original plan was to read the phone’s battery charge to determine when the power should disconnect, but that proved to be rather difficult so we opted for a simple timer based disconnect instead.



Once the phone connects to the device, the user gains access to the web app which lets them set a time in hours and minutes. When pressing “Start Timer”, the set time is sent to the ESP32 to initiate the countdown. This makes it so that once the time is set, the device will continue functioning without the phone itself, meaning the user can turn off their phone and the device will disconnect the power at the set time regardless.

```
void setup() {
  myStepper.setSpeed(60);
  Serial.begin(115200);
  Serial.begin(9600);

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.println("Connecting to WiFi...");
  }

  Serial.print("ESP32 IP Address: ");
  Serial.println(WiFi.localIP());

  server.on("/", HTTP_GET, [](AsyncWebServerRequest *request) {
    request->send_P(200, "text/html", index_html);
  });

  server.on("/setTimer", HTTP_GET, [](AsyncWebServerRequest *request) {
    if (request->hasParam("seconds")) {
      timerDuration = request->getParam("seconds")->value().toInt();
      timerStartMillis = millis();
      timerRunning = true;

      Serial.print("Timer started for ");
      Serial.print(timerDuration);
      Serial.println(" seconds");

      request->send(200, "text/plain", "OK");
    } else {
      request->send(400, "text/plain", "Missing parameter");
    }
  });

  server.on("/timeRemaining", HTTP_GET, [](AsyncWebServerRequest *request) {
    if (!timerRunning) {
      request->send(200, "text/plain", "0");
      return;
    }

    unsigned long elapsed = (millis() - timerStartMillis) / 1000;

    if (elapsed >= timerDuration) {
      request->send(200, "text/plain", "0");
    } else {
      request->send(200, "text/plain",
        String(timerDuration - elapsed));
    }
  });

  server.begin();
}
```

The built version of the prototype consists of a cardboard box with a built-in trapdoor to support the wireless charging pad.



The wireless charging pad is continuously plugged in, but since these pads only have an effective charging range of ~3 to 5 mm, simply lowering it a bit will cause the phone to stop charging

The trapdoor is held up by a stepper motor with a lever. Once the time set by the web app has elapsed, the lever will rotate out of the way and cause the charger to drop, creating more than enough clearance to stop the charging process.

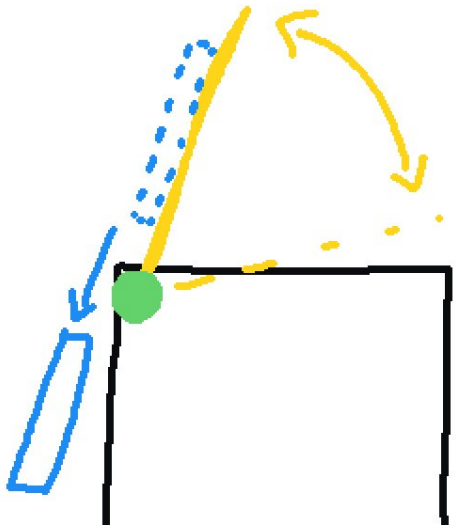
```
void loop() {
  if (timerRunning) {
    unsigned long elapsed = (millis() - timerStartMillis) / 1000;
    if (elapsed >= timerDuration) {
      timerRunning = false;
      onTimerFinished();
    }
  }
}

void onTimerFinished() {
  Serial.println("TIMER DONE!");

  myStepper.step(stepsPerRevolution);
  delay(500);
  myStepper.step(-stepsPerRevolution);
}
```

Once the sequence is complete, the device has to be manually reset by pushing up the trapdoor again. The lever can be instantly rotated by using the web app and setting the time to 0h-0m.

Project insights

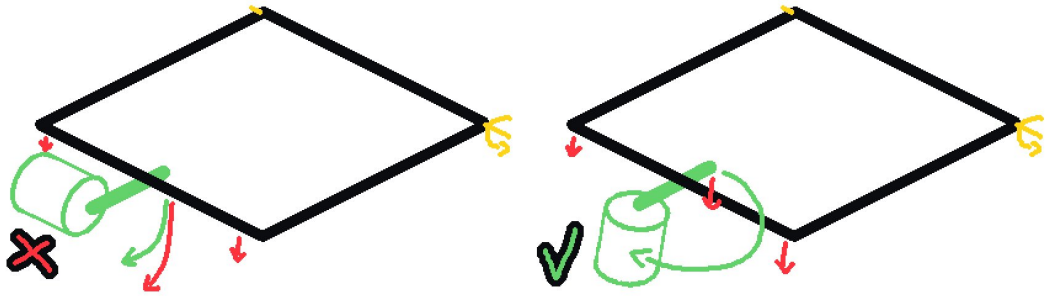
Probably the most significant lesson learned form this project is the fact that when you want to be able to move parts, you should try to let gravity do as much work as possible and have as few moving parts as possible.

In our case the primary issue with the designs was usually the amount of force we could create with the tools we had (specifically the stepper motor). One of the earliest ideas for the device was to have the platform with the charging pad lift the phone away and off the device.



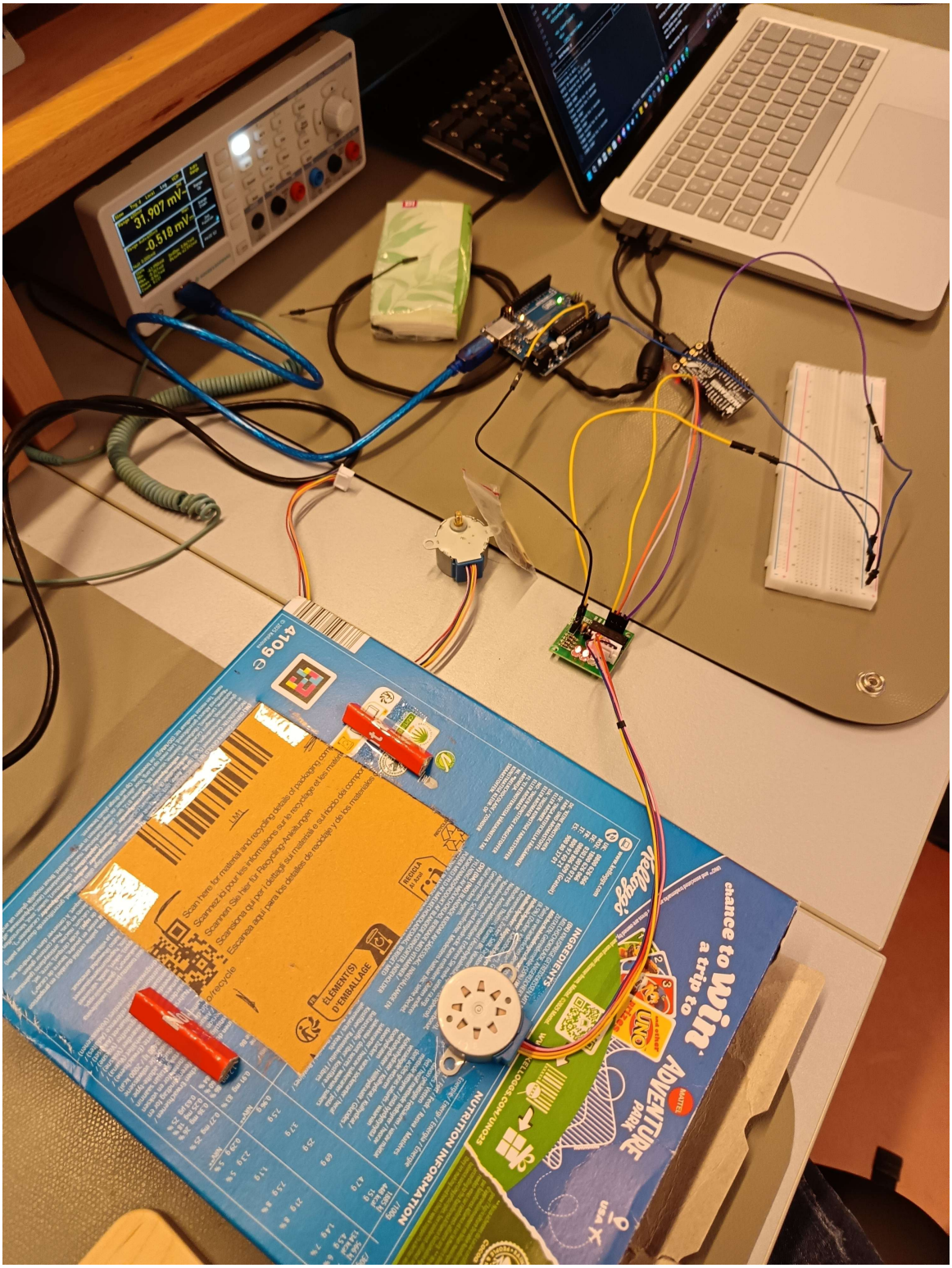
But doing this requires a lot of torque which we weren't sure we really even had with the motors available. Additionally, using multiple motors could lead to asynchronous execution and make the whole thing unreliable, not to mention requiring even more power.

Another thing we decided to do was to implement the motor in a way where the weight of the trapdoor rests on the motor in a way that the force cannot shift the position of the rotor in a way that isn’t intended.



While figure left could maybe permit an easier implementation of an automatic device reset, figure right (which we used) made the whole system more reliable short term, which was favorable for the prototype we were creating.

Images and Demo Video



Prototype device with wires exposed. The arduino Uno in the image does not run any code, in this instance it is purely used as a power supply for the stepper motor.

Device with phone resting on the intended support structure.

The inside of the Device, showing the lever holding the trapdoor in place.

Video Demonstration of device can be viewed here:

https://youtu.be/kG8yg9UVM_g